

maze_bfs

Name:Asmit Sahu roll:23052231

```
In [3]: def bfs_maze(maze, start, goal):
        rows, cols = len(maze), len(maze[0])

        visited = set()
        queue = [(start, [start])]
        visited.add(start)

        while queue:
            (x, y), path = queue.pop(0)

            if (x, y) == goal:
                return path

            for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
                nx, ny = x + dx, y + dy

                if (0 <= nx < rows and
                    0 <= ny < cols and
                    maze[nx][ny] == 0 and
                    (nx, ny) not in visited):

                    visited.add((nx, ny))
                    queue.append(((nx, ny), path + [(nx, ny)]))

        return None
```

```
In [4]: maze = [
        [0, 0, 1, 0, 0],
        [1, 0, 1, 0, 1],
        [0, 0, 0, 0, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0]
        ]

        start = (0, 0)
        goal = (4, 3)

        print(bfs_maze(maze, start, goal))
```

```
[(0, 0), (0, 1), (1, 1), (2, 1), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3)]
```

maze_dfs

Name:Asmit Sahu roll:23052231

```
In [5]: def dfs_maze(maze, start, goal):
        rows, cols = len(maze), len(maze[0])
        visited = set()

        stack = [(start, [start])]
        visited.add(start)

        while stack:
            (x, y), path = stack.pop()

            if (x, y) == goal:
                return path

            for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
                nx, ny = x + dx, y + dy

                if (0 <= nx < rows and
                    0 <= ny < cols and
                    maze[nx][ny] == 0 and
                    (nx, ny) not in visited):

                    visited.add((nx, ny))
                    stack.append(((nx, ny), path + [(nx, ny)]))

        return None
```

```
In [6]: maze = [
        [0, 0, 1, 0, 0],
        [1, 0, 1, 0, 1],
        [0, 0, 0, 0, 0],
        [0, 1, 1, 1, 0],
        [0, 0, 0, 0, 0]
    ]

    start = (0, 0)
    goal = (4, 3)

    print(dfs_maze(maze, start, goal))

[(0, 0), (0, 1), (1, 1), (2, 1), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4), (4, 3)]
```